

AI/BigData 인텐시브 과정

강사장철원

Section 0

코스 소개

DAY1	DAY2	DAY3	DAY4	DAY5
데이터 분석을 위한 기초 수학			데이터 집계 및 시각화	
DAY6	DAY7	DAY8	DAY9	DAY10
	미니 프로젝트	확률분포, AB테스트		
DAY11	DAY12	DAY13	DAY14	DAY15
머신러닝				
DAY16	DAY17	DAY18	DAY19	DAY20
딥러닝		파이널 프로젝트		

□ 결측치 처리

□ 데이터 결합

□ 새로운 열 생성

□ 날짜 처리

□ groupby, crosstab, pivot_table

□ one hot encoding

AI/BigData 인텐시브 과정

Section 1. 데이터 전처리

Section 1-1. 데이터 살펴보기

데이터 불러오기

In [1]: ▶

```
1 import numpy as np
2 import pandas as pd
```

In [2]: ▶

```
1 df1 = pd.read_csv('./data/sales1.csv', encoding='cp949')
2 df2 = pd.read_csv('./data/sales2.csv', encoding='cp949')
3 df3 = pd.read_csv('./data/sales3.csv', encoding='cp949')
```

데이터 확인

In [3]: ▶

1 df1.head(3)

Out[3]:

	ORDERID	PURCHASE_PAGE	CITY	CUSTOMER_TYPE	GENDER
0	750-67-8428	A	Seoul	Member	Female

1 226-31-3081

2 631-41-3108

In [4]: ▶

1 df2.head(3)

Out[4]:

	ORDERID	PRODUCT_TYPE	UNIT_PRICE	QUANTITY	MILEAGE	TOTAL_PRICE
0	750-67-8428	Health and beauty	74.69	7	26.1415	548.9715
1	226-31-3081	Electronic accessories	15.28	5	3.8200	80.2200

2 631-41-3108

In [5]: ▶

1 df3.head(3)

Out[5]:

	ORDERID	DATE	PURCHASE_TIME	PAYMENT	COGS	GROSS_MARGIN_PERCENTAGE	GROSS_INCOME	RATING
0	750-67-8428	1/5/2019	13:08	Samsungpay	522.83	4.761905	26.1415	9.1
1	226-31-3081	3/8/2019	10:29	Naverpay	76.40	4.761905	3.8200	9.6
2	631-41-3108	3/3/2019	13:23	Credit card	324.31	4.761905	16.2155	7.4

유니크 값 확인

```
In [6]: 1 df1['PURCHASE_PAGE'].unique()
```

```
Out[6]: array(['A', 'C', 'B'], dtype=object)
```

```
In [7]: 1 df1['CITY'].unique()
```

```
Out[7]: array(['Seoul', 'Gyeonggi', 'Incheon'], dtype=object)
```

```
In [8]: 1 df1['CUSTOMER_TYPE'].unique()
```

```
Out[8]: array(['Member', 'Normal'], dtype=object)
```

```
In [9]: 1 df1['GENDER'].unique()
```

```
Out[9]: array(['Female', 'Male'], dtype=object)
```

```
In [10]: 1 df3['RATING'].unique()
```

```
Out[10]: array([ 9.1,  9.6,  7.4,  8.4, nan,  4.1,  5.8,  8. ,  7.2,  5.9,  4.5,
                  6.8,  7.1,  8.2,  5.7,  4.6,  6.9,  8.6,  4.4,  4.8,  5.1,  9.9,
                  6. ,  8.5,  6.7,  7.7,  7.5,  7. ,  4.7,  7.6,  7.9,  6.3,  5.6,
                  9.5,  8.1,  6.5,  6.1,  6.6,  5.4,  9.3, 10. ,  6.4,  4.3,  4. ,
                  8.7,  9.4,  5.5,  8.3,  7.3,  4.9,  4.2,  9.2,  7.8,  5.3,  5.2,
                  9. ,  8.8,  6.2,  9.8,  9.7,  5. ,  8.9])
```

알면 도움되는 데이터 확인 방법

```
In [11]: 1 df1['GENDER'].unique()
```

```
Out[11]: array(['Female', 'Male'], dtype=object)
```

```
In [12]: 1 df1['GENDER'].nunique()
```

```
Out[12]: 2
```

```
In [13]: 1 df1['GENDER'].value_counts()
```

```
Out[13]: Female    501  
         Male      499  
         Name: GENDER, dtype: int64
```

AI/BigData 인텐시브 과정

Section 1. 데이터 전처리

Section 1-2. 결측치 처리

결측치 확인

null인가? 합계

```
In [14]: ▶ 1 df3.isnull().sum()
```

```
Out[14]: ORDERID  
         DATE  
         PURCHASE_TIME  
         PAYMENT  
         COGS  
         GROSS_MARGIN_PERCENTAGE  
         GROSS_INCOME  
         RATING  
         dtype: int64
```

0
0
0
0
0
0
0
4

결측치 확인

null인가?

```
In [15]: ▶ 1 df3[df3['RATING'].isnull()==True]
```

Out[15]:

	ORDERID	DATE	PURCHASE_TIME	PAYMENT	COGS	GROSS_MARGIN_PERCENTAGE	GROSS_INCOME	RATING
4	373-73-7910	2/8/2019	10:37	Samsungpay	604.17	4.761905	30.2085	NaN
228	466-61-5506	2/26/2019	10:52	Naverpay	544.20	4.761905	27.2100	NaN
517	521-18-7827	1/22/2019	20:46	Credit card	196.95	4.761905	9.8475	NaN
701	176-64-7711	3/27/2019	19:11	Credit card	96.96	4.761905	4.8480	NaN

최솟값, 최댓값 확인

In [16]:



```
1 min(df3['RATING'])
```

Out[16]: 4.0

In [17]:



```
1 max(df3['RATING'])
```

Out[17]: 10.0

결측치 0으로 채우기

```
In [19]: ▶ 1 df3 = df3.fillna(0)
```

```
In [20]: ▶ 1 df3.head(5)
```

Out[20]:

	ORDERID	DATE	PURCHASE_TIME	PAYMENT	COGS	GROSS_MARGIN_PERCENTAGE	GROSS_INCOME	RATING
0	750-67-8428	1/5/2019	13:08	Samsungpay	522.83	4.761905	26.1415	9.1
1	226-31-3081	3/8/2019	10:29	Naverpay	76.40	4.761905	3.8200	9.6
2	631-41-3108	3/3/2019	13:23	Credit card	324.31	4.761905	16.2155	7.4
3	123-19-1176	1/27/2019	20:33	Samsungpay	465.76	4.761905	23.2880	8.4
4	373-73-7910	2/8/2019	10:37	Samsungpay	604.17	4.761905	30.2085	0.0

결측치 확인

In [21]: ▶

```
1 df3.isnull().sum()
```

```
Out[21]: ORDERID          0  
         DATE            0  
         PURCHASE_TIME   0  
         PAYMENT         0  
         COGS            0  
         GROSS_MARGIN_PERCENTAGE  0  
         GROSS_INCOME    0  
         RATING          0  
         dtype: int64
```

In [22]: ▶

```
1 df3[df3['RATING'].isnull()==True]
```

Out[22]:

ORDERID	DATE	PURCHASE_TIME	PAYMENT	COGS	GROSS_MARGIN_PERCENTAGE	GROSS_INCOME	RATING
---------	------	---------------	---------	------	-------------------------	--------------	--------

공백 제거

```
In [22]: 1 df3[df3['RATING'].isnull()==True]
```

Out[22]:

ORDERID	DATE	PURCHASE_TIME	PAYMENT	COGS	GROSS_MARGIN_PERCENTAGE	GROSS_INCOME	RATING
---------	------	---------------	---------	------	-------------------------	--------------	--------

```
In [23]: 1 df3['PAYMENT'].unique()
```

Out[23]: array(['Samsungpay', 'Naverpay', 'Credit card', 'Samsungpay '],
dtype=object)

```
In [24]: 1 df3['PAYMENT'] = df3['PAYMENT'].str.strip()
```

```
In [25]: 1 df3['PAYMENT'].unique()
```

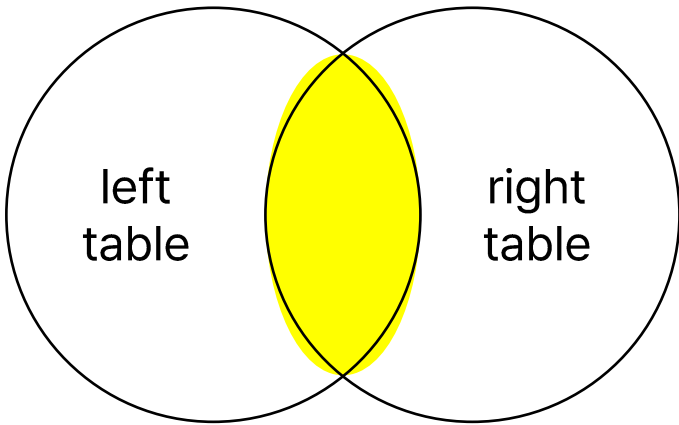
Out[25]: array(['Samsungpay', 'Naverpay', 'Credit card'], dtype=object)

AI/BigData 인텐시브 과정

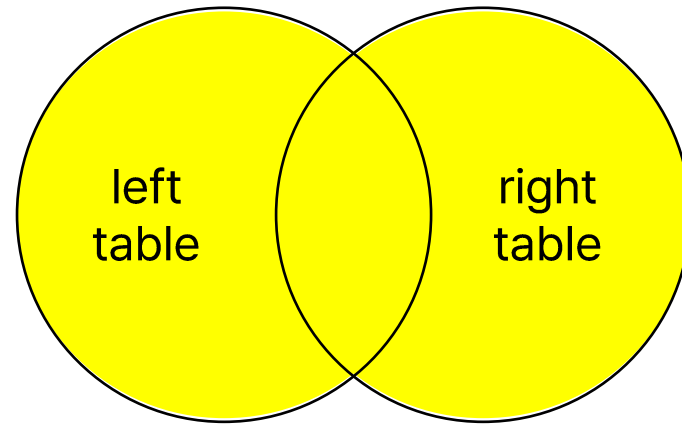
Section 1. 데이터 전처리

Section 1-3. 데이터 결합

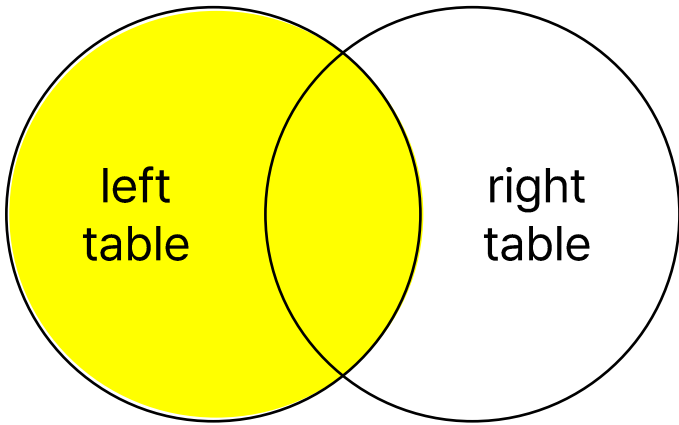
데이터 join



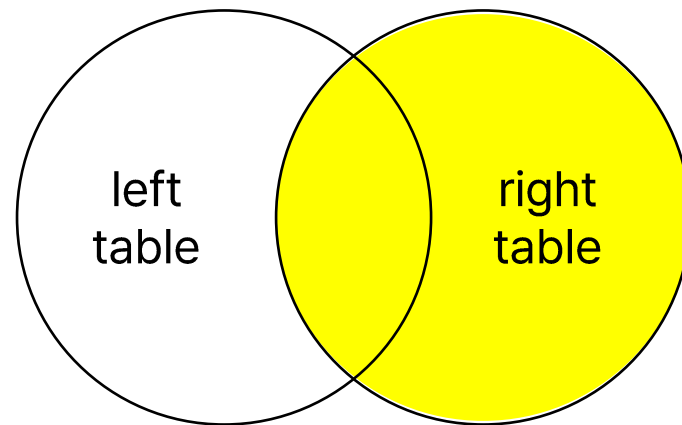
inner join



full join

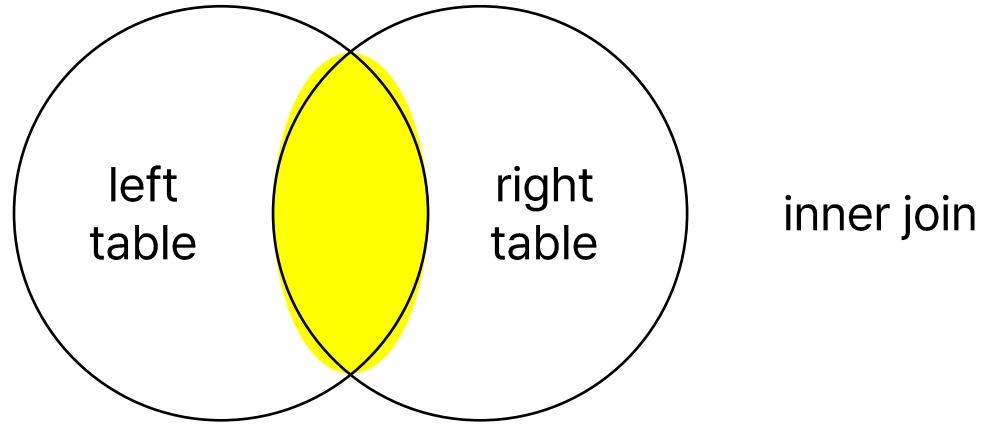


left join



right join

inner join



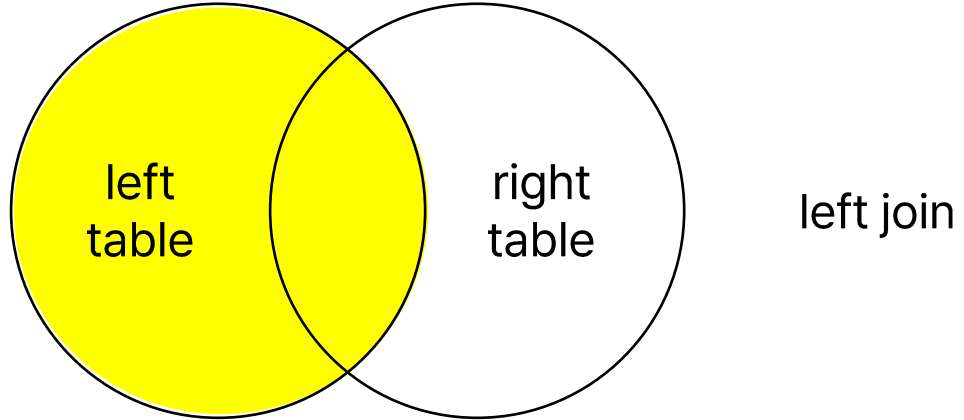
A	B
1	책상
2	의자
3	키보드
4	모니터
5	노트북

A	C
3	100
4	500
5	800
6	600
7	300



A	B	C
3	키보드	100
4	모니터	500
5	노트북	800

left join



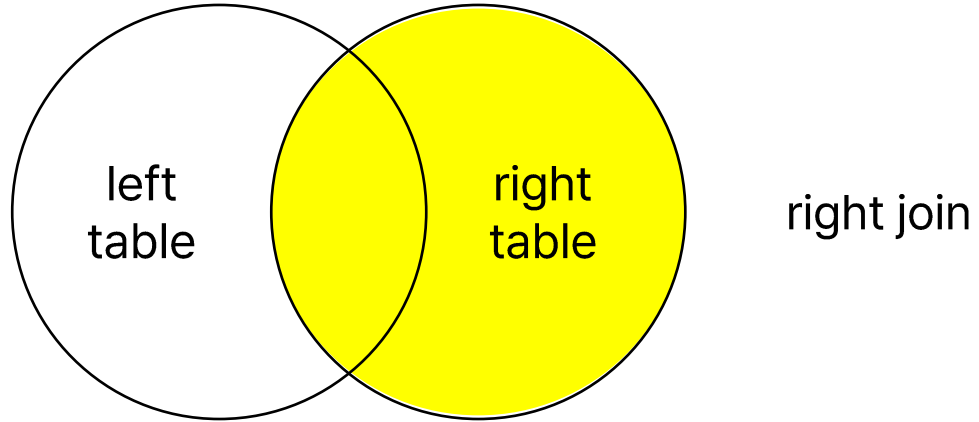
A	B
1	책상
2	의자
3	키보드
4	모니터
5	노트북

A	C
3	100
4	500
5	800
6	600
7	300



A	B	C
1	책상	
2	의자	
3	키보드	100
4	모니터	500
5	노트북	800

right join



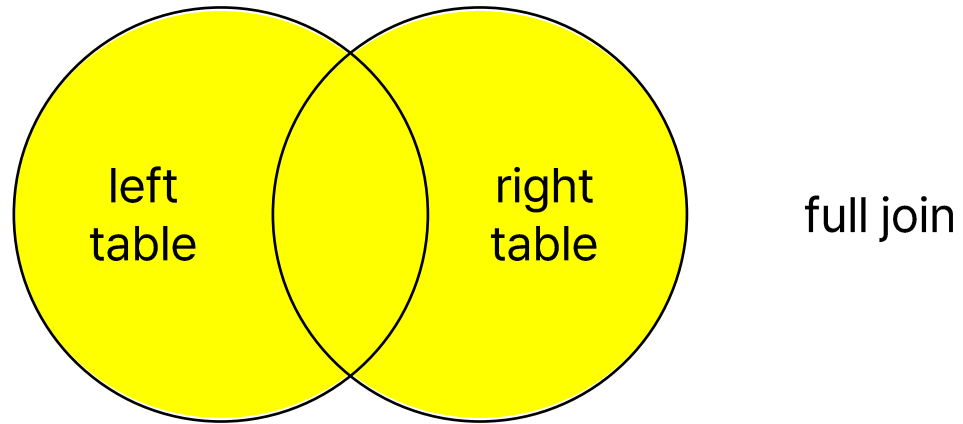
A	B
1	책상
2	의자
3	키보드
4	모니터
5	노트북

A	C
3	100
4	500
5	800
6	600
7	300



A	B	C
3	키보드	100
4	모니터	500
5	노트북	800
6		600
7		300

full join



A	B
1	책상
2	의자
3	키보드
4	모니터
5	노트북

A	C
3	100
4	500
5	800
6	600
7	300



A	B	C
1	책상	
2	의자	
3	키보드	100
4	모니터	500
5	노트북	800
6		600
7		300

inner join

In [26]: ▶

1df1

Out[26]:

	ORDERID	PURCHASE_PAGE	CITY	CUSTOMER_TYPE	GENDER
0	750-67-8428	A	Seoul	Member	Female
1	226-31-3081	C	Gyeonggi	Normal	Female
2	631-41-3108	A	Seoul	Normal	Male
3	123-19-1176	A	Seoul	Member	Male
4	373-73-7910	A	Seoul	Normal	Male
...	...	...	...	...	...
995	233-67-5758	C	Gyeonggi	Normal	Male
996	303-96-2227	B	Incheon	Normal	Female
997	727-02-1313	A	Seoul	Member	Male
998	347-56-2442	A	Seoul	Normal	Male
999	849-09-3807	A	Seoul	Member	Female

1000 rows × 5 columns

In [27]: ▶

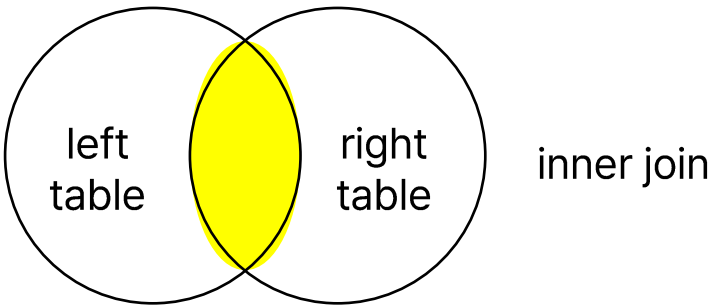
1df2

Out[27]:

	ORDERID	PRODUCT_TYPE	UNIT_PRICE	QUANTITY	MILEAGE	TOTAL_PRICE
0	750-67-8428	Health and beauty	74.69	7	26.1415	548.9715
1	226-31-3081	Electronic accessories	15.28	5	3.8200	80.2200
2	631-41-3108	Home and lifestyle	46.33	7	16.2155	340.5255
3	123-19-1176	Health and beauty	58.22	8	23.2880	489.0480
4	373-73-7910	Sports and travel	86.31	7	30.2085	634.3785
...	...	...	...	...	...	...
995	233-67-5758	Health and beauty	40.35	1	2.0175	42.3675
996	303-96-2227	Home and lifestyle	97.38	10	48.6900	1022.4900
997	727-02-1313	Food and beverages	31.84	1	1.5920	33.4320
998	347-56-2442	Home and lifestyle	65.82	1	3.2910	69.1110
999	849-09-3807	Fashion accessories	88.34	7	30.9190	649.2990

1000 rows × 6 columns

inner join



기준 열 병합 방법

```
In [28]: ▶ 1 df12_inner = pd.merge(df1, df2, on='ORDERID', how='inner')
```

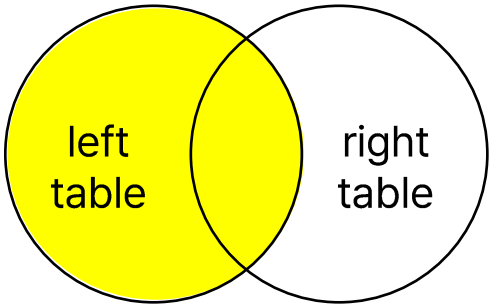
```
In [29]: ▶ 1 df12_inner
```

Out[29]:

	ORDERID	PURCHASE_PAGE	CITY	CUSTOMER_TYPE	GENDER	PRODUCT_TYPE	UNIT_PRICE	QUANTITY	MILEAGE	TOTAL_PRICE
0	750-67-8428	A	Seoul	Member	Female	Health and beauty	74.69	7	26.1415	548.9715
1	226-31-3081	C	Gyeonggi	Normal	Female	Electronic accessories	15.28	5	3.8200	80.2200
2	631-41-3108	A	Seoul	Normal	Male	Home and lifestyle	46.33	7	16.2155	340.5255
3	123-19-1176	A	Seoul	Member	Male	Health and beauty	58.22	8	23.2880	489.0480
4	373-73-7910	A	Seoul	Normal	Male	Sports and travel	86.31	7	30.2085	634.3785
...	...	...	...	...	...	...	...	...	...	...
995	233-67-5758	C	Gyeonggi	Normal	Male	Health and beauty	40.35	1	2.0175	42.3675
996	303-96-2227	B	Incheon	Normal	Female	Home and lifestyle	97.38	10	48.6900	1022.4900
997	727-02-1313	A	Seoul	Member	Male	Food and beverages	31.84	1	1.5920	33.4320
998	347-56-2442	A	Seoul	Normal	Male	Home and lifestyle	65.82	1	3.2910	69.1110
999	849-09-3807	A	Seoul	Member	Female	Fashion accessories	88.34	7	30.9190	649.2990

1000 rows x 10 columns

left join



left join

```
In [30]: 1 df12_left = pd.merge(df1, df2, on='ORDERID', how='left')
```

```
In [31]: 1 df12_left
```

Out[31]:

	ORDERID	PURCHASE_PAGE	CITY	CUSTOMER_TYPE	GENDER	PRODUCT_TYPE	UNIT_PRICE	QUANTITY	MILEAGE	TOTAL_PRICE
0	750-67-8428	A	Seoul	Member	Female	Health and beauty	74.69	7	26.1415	548.9715
1	226-31-3081	C	Gyeonggi	Normal	Female	Electronic accessories	15.28	5	3.8200	80.2200
2	631-41-3108	A	Seoul	Normal	Male	Home and lifestyle	46.33	7	16.2155	340.5255
3	123-19-1176	A	Seoul	Member	Male	Health and beauty	58.22	8	23.2880	489.0480
4	373-73-7910	A	Seoul	Normal	Male	Sports and travel	86.31	7	30.2085	634.3785
...	...	...	...	...	...	...	...	...	...	...
995	233-67-5758	C	Gyeonggi	Normal	Male	Health and beauty	40.35	1	2.0175	42.3675
996	303-96-2227	B	Incheon	Normal	Female	Home and lifestyle	97.38	10	48.6900	1022.4900
997	727-02-1313	A	Seoul	Member	Male	Food and beverages	31.84	1	1.5920	33.4320
998	347-56-2442	A	Seoul	Normal	Male	Home and lifestyle	65.82	1	3.2910	69.1110
999	849-09-3807	A	Seoul	Member	Female	Fashion accessories	88.34	7	30.9190	649.2990

1000 rows x 10 columns

df1 + df2 + df3

In [31]: ▶

1 df12_left

Out[31]:

	ORDERID	PURCHASE_PAGE	CITY	CUSTOMER_TYPE	GENDER	PRODUCT_TYPE	UNIT_PRICE	QUANTITY	MILEAGE	TOTAL_PRICE
0	750-67-8428	A								
1	226-31-3081	C								
2	631-41-3108	A								
3	123-19-1176	A								
4	373-73-7910	A								
...	...	...								
995	233-67-5758	C								
996	303-96-2227	B								
997	727-02-1313	A								
998	347-56-2442	A								
999	849-09-3807	A								

1000 rows x 10 columns

In [32]: ▶

1 df3

Out[32]:

	ORDERID	DATE	PURCHASE_TIME	PAYMENT	COGS	GROSS_MARGIN_PERCENTAGE	GROSS_INCOME	RATING
0	750-67-8428	1/5/2019	13:08	Samsungpay	522.83	4.761905	26.1415	9.1
1	226-31-3081	3/8/2019	10:29	Naverpay	76.40	4.761905	3.8200	9.6
2	631-41-3108	3/3/2019	13:23	Credit card	324.31	4.761905	16.2155	7.4
3	123-19-1176	1/27/2019	20:33	Samsungpay	465.76	4.761905	23.2880	8.4
4	373-73-7910	2/8/2019	10:37	Samsungpay	604.17	4.761905	30.2085	0.0
...	...	...	...	...	...	...	...	...
995	233-67-5758	1/29/2019	13:46	Samsungpay	40.35	4.761905	2.0175	6.2
996	303-96-2227	3/2/2019	17:16	Samsungpay	973.80	4.761905	48.6900	4.4
997	727-02-1313	2/9/2019	13:22	Naverpay	31.84	4.761905	1.5920	7.7
998	347-56-2442	2/22/2019	15:33	Naverpay	65.82	4.761905	3.2910	4.1
999	849-09-3807	2/18/2019	13:28	Naverpay	618.38	4.761905	30.9190	6.6

1000 rows x 8 columns

Section

데이터 결합하기

df1 + df2 + df3

In [34]:

1 df = pd.merge(df12_left, df3, on='ORDERID', how='left')

In [35]:

1 df

Out[35]:

	ORDERID	PURCHASE_PAGE	CITY	CUSTOMER_TYPE	GENDER	PRODUCT_TYPE	UNIT_PRICE	QUANTITY	MILEAGE	TOTAL_PRICE	DATE	PURCHASE_TIME
0	750-67-8428	A	Seoul	Member	Female	Health and beauty	74.69	7	26.1415	548.9715	1/5/2019	13:08
1	226-31-3081	C	Gyeonggi	Normal	Female	Electronic accessories	15.28	5	3.8200	80.2200	3/8/2019	10:29
2	631-41-3108	A	Seoul	Normal	Male	Home and lifestyle	46.33	7	16.2155	340.5255	3/3/2019	13:23
3	123-19-1176	A	Seoul	Member	Male	Health and beauty	58.22	8	23.2880	489.0480	1/27/2019	20:33
4	373-73-7910	A	Seoul	Normal	Male	Sports and travel	86.31	7	30.2085	634.3785	2/8/2019	10:37
...	...	...	...	...	...	...	...	...	...	...	...	...
995	233-67-5758	C	Gyeonggi	Normal	Male	Health and beauty	40.35	1	2.0175	42.3675	1/29/2019	13:46
996	303-96-2227	B	Incheon	Normal	Female	Home and lifestyle	97.38	10	48.6900	1022.4900	3/2/2019	17:16
997	727-02-1313	A	Seoul	Member	Male	Food and beverages	31.84	1	1.5920	33.4320	2/9/2019	13:22
998	347-56-2442	A	Seoul	Normal	Male	Home and lifestyle	65.82	1	3.2910	69.1110	2/22/2019	15:33
999	849-09-3807	A	Seoul	Member	Female	Fashion accessories	88.34	7	30.9190	649.2990	2/18/2019	13:28

1000 rows x 17 columns

데이터 저장

csv파일로 저장

In [36]:



1

```
df.to_csv('./data/sales.csv', index=False)
```



이름



house_prices.csv



sales3.csv



sales1.csv



sales2.csv



sales.csv

Section 데이터 결합하기

concat

```
In [53]: 1 df001 = pd.DataFrame([[ '철원', 3],  
2                               [ '혁진', 7]],  
3                               columns=[ 'name', 'number'])
```

```
In [54]: 1 df001
```

column 이름 설정

Out[54]:

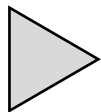
	name	number
0	철원	3
1	혁진	7

```
In [55]: 1 df002 = pd.DataFrame([[ '예인', 9],  
2                               [ '솔이', 8]],  
3                               columns=[ 'name', 'number'])
```

```
In [56]: 1 df002
```

Out[56]:

	name	number
0	예인	9
1	솔이	8



```
In [57]: 1 df003 = pd.concat([df001, df002])
```

```
In [58]: 1 df003
```

Out[58]:

	name	number
0	철원	3
1	혁진	7
0	예인	9
1	솔이	8

```
In [59]: 1 df004 = df003.reset_index()
```

인덱스 초기화

```
In [60]: 1 df004
```

Out[60]:

	index	name	number
0	0	철원	3
1	1	혁진	7
2	0	예인	9
3	1	솔이	8

concat

In [61]: ▶

```
1 df005 = pd.concat([df001, df002], ignore_index=True)
```

인덱스 무시

In [62]: ▶

```
1 df005
```

Out [62]:

	name	number
0	철원	3
1	혁진	7
2	예인	9
3	솔이	8

concat

```
In [63]: 1 df001 = pd.DataFrame([[ '철원', 3],
2                               [ '혁진', 7]],
3                               columns=[ 'name', 'number'])
```

```
In [64]: 1 df001
```

Out[64]:

	name	number
0	철원	3
1	혁진	7

```
In [65]: 1 df006 = pd.DataFrame([[ '피아노', '강사'],
2                               [ '노래', '매니저']],
3                               columns=[ 'hobby', 'job'])
```

```
In [66]: 1 df006
```

Out[66]:

	hobby	job
0	피아노	강사
1	노래	매니저

```
In [67]: 1 df007 = pd.concat([df001, df006], axis=1)
```

```
In [68]: 1 df007
```

Out[68]:

	name	number	hobby	job
0	철원	3	피아노	강사
1	혁진	7	노래	매니저

AI/BigData 인텐시브 과정

Section 1. 데이터 전처리

Section 1-4. 데이터 조작

데이터 열(column) 추출

```
In [69]: 1 df = pd.read_csv('./data/sales.csv', encoding='cp949')
```

```
In [70]: 1 df.head(5)
```

Out[70]:

	ORDERID	PURCHASE_PAGE	CITY	CUSTOMER_TYPE	GENDER	PRODUCT_TYPE	UNIT_PRICE	QUANTITY	MILEAGE	TOTAL_PRICE	DATE	PURCHASE_TIME	
0	750-67-8428	A	Seoul	Member	Female	Health and beauty	74.69	7	26.1415	548.9715	1/5/2019	13:08	Sa
1	226-31-3081	C	Gyeonggi	Normal	Female	Electronic accessories	15.28	5	3.8200	80.2200	3/8/2019	10:29	
2	631-41-3108	A	Seoul	Normal	Male	Home and lifestyle	46.33	7	16.2155	340.5255	3/3/2019	13:23	
3	123-19-1176	A	Seoul	Member	Male	Health and beauty	58.22	8	23.2880	489.0480	1/27/2019	20:33	Sa
4	373-73-7910	A	Seoul	Normal	Male	Sports and travel	86.31	7	30.2085	634.3785	2/8/2019	10:37	Sa

데이터 열(column) 추출

```
In [71]: 1 df008 = df[['UNIT_PRICE', 'QUANTITY']]
```

```
In [72]: 1 df008
```

추출할 열

Out[72]:

	UNIT_PRICE	QUANTITY
0	74.69	7
1	15.28	5
2	46.33	7
3	58.22	8
4	86.31	7
...	...	...
995	40.35	1
996	97.38	10
997	31.84	1
998	65.82	1
999	88.34	7

1000 rows x 2 columns

데이터 행(row) 추출

In [73]:

▶

1 df.head(5)

Out[73]:

	ORDERID	PURCHASE_PAGE	CITY	CUSTOMER_TYPE	GENDER	PRODUCT_TYPE	UNIT_PRICE	QUANTITY	MILEAGE	TOTAL_PRICE	DATE	PURCHASE_TIME
0	750-67-8428	A	Seoul	Member	Female	Health and beauty	74.69	7	26.1415	548.9715	1/5/2019	13:08
1	226-31-3081	C	Gyeonggi	Normal	Female	Electronic accessories	15.28	5	3.8200	80.2200	3/8/2019	10:29
2	631-41-3108	A	Seoul	Normal	Male	Home and lifestyle	46.33	7	16.2155	340.5255	3/3/2019	13:23
3	123-19-1176	A	Seoul	Member	Male	Health and beauty	58.22	8	23.2880	489.0480	1/27/2019	20:33
4	373-73-7910	A	Seoul	Normal	Male	Sports and travel	86.31	7	30.2085	634.3785	2/8/2019	10:37

Section

데이터 조작

UNIT_PRICE > 50인 행만 추출

```
In [74]: 1 unit = df[df['UNIT_PRICE'] > 50]
```

```
In [75]: 1 unit
```

Out[75]:

	ORDERID	PURCHASE_PAGE	CITY	CUSTOMER_TYPE	GENDER	PRODUCT_TYPE	UNIT_PRICE	QUANTITY
0	750-67-8428	A	Seoul	Member	Female	Health and beauty	74.69	7
3	123-19-1176	A	Seoul	Member	Male	Health and beauty	58.22	8
4	373-73-7910	A	Seoul	Normal	Male	Sports and travel	86.31	7
5	699-14-3026	C	Gyeonggi	Normal	Male	Electronic accessories	85.39	7
6	355-53-5943	A	Seoul	Member	Female	Electronic accessories	68.84	6
...	...	...	...	...	...	...	...	...
992	745-74-0715	A	Seoul	Normal	Male	Electronic accessories	58.03	2
994	652-49-6720	C	Gyeonggi	Member	Female	Electronic accessories	60.95	1
996	303-96-2227	B	Incheon	Normal	Female	Home and lifestyle	97.38	10
998	347-56-2442	A	Seoul	Normal	Male	Home and lifestyle	65.82	1
999	849-09-3807	A	Seoul	Member	Female	Fashion accessories	88.34	7

561 rows x 17 columns

```
In [76]: 1 unit_ratio1 = len(unit)/len(df)
2 print(unit_ratio1)
```

0.561

```
In [77]: 1 unit_ratio2 = unit.shape[0]/df.shape[0]
2 print(unit_ratio2)
```

0.561

```
In [78]: 1 unit.shape[0]
```

Out[78]: 561

Section

데이터 조작

도시가 서울에 해당하는 행만 추출

```
In [79]: 1 seoul = df[df['CITY']=='Seoul']
```

```
In [80]: 1 seoul
```

Out[80]:

	ORDERID	PURCHASE_PAGE	CITY	CUSTOMER_TYPE	GENDER	PRODUCT_TYPE	UNIT_PRICE	QUANTITY
0	750-67-8428	A	Seoul	Member	Female	Health and beauty	74.69	7
2	631-41-3108	A	Seoul	Normal	Male	Home and lifestyle	46.33	7
3	123-19-1176	A	Seoul	Member	Male	Health and beauty	58.22	8
4	373-73-7910	A	Seoul	Normal	Male	Sports and travel	86.31	7
6	355-53-5943	A	Seoul	Member	Female	Electronic accessories	68.84	6
...	...	...	...	...	...	...	...	...
990	886-18-2897	A	Seoul	Normal	Female	Food and beverages	56.56	5
992	745-74-0715	A	Seoul	Normal	Male	Electronic accessories	58.03	2
997	727-02-1313	A	Seoul	Member	Male	Food and beverages	31.84	1
998	347-56-2442	A	Seoul	Normal	Male	Home and lifestyle	65.82	1
999	849-09-3807	A	Seoul	Member	Female	Fashion accessories	88.34	7

340 rows x 17 columns

```
In [81]: 1 seoul_ratio1 = len(seoul)/len(df)
2 print(seoul_ratio1)
```

0.34

```
In [82]: 1 seoul_ratio2 = seoul.shape[0]/df.shape[0]
2 print(seoul_ratio2)
```

0.34

두 조건을 동시에 만족하는 데이터 행 추출

```
In [83]: 1 unit_seoul = df[(df['UNIT_PRICE']>50)&(df['CITY']=='Seoul')]
```

```
In [84]: 1 unit_seoul.head(5)
```

Out[84]:

	ORDERID	PURCHASE_PAGE	CITY	CUSTOMER_TYPE	GENDER	PRODUCT_TYPE	UNIT_PRICE	QUANTITY	MILEAGE	TOTAL_PRICE
0	750-67-8428	A	Seoul	Member	Female	Health and beauty	74.69	7	26.1415	548.9715
3	123-19-1176	A	Seoul	Member	Male	Health and beauty	58.22	8	23.2880	489.0480
4	373-73-7910	A	Seoul	Normal	Male	Sports and travel	86.31	7	30.2085	634.3785
6	355-53-5943	A	Seoul	Member	Female	Electronic accessories	68.84	6	20.6520	433.6920
14	829-34-3910	A	Seoul	Normal	Female	Health and beauty	71.38	10	35.6900	749.4900

```
In [85]: 1 unit_seoul.shape
```

Out[85]: (189, 17)

데이터 행, 열 일부 추출(iloc, loc)

1 df.head(10)

36]:

	ORDERID	PURCHASE_PAGE	CITY	CUSTOMER_TYPE	GENDER	PRODUCT_TYPE	UNIT_PRICE	QUANTITY
0	750-67-8428	A	Seoul	Member	Female	Health and beauty	74.69	7
1	226-31-3081	C	Gyeonggi	Normal	Female	Electronic accessories	15.28	5
2	631-41-3108	A	Seoul	Normal	Male	Home and lifestyle	46.33	7
3	123-19-1176	A	Seoul	Member	Male	Health and beauty	58.22	8
4	373-73-7910	A	Seoul	Normal	Male	Sports and travel	86.31	7
5	699-14-3026	C	Gyeonggi	Normal	Male	Electronic accessories	85.39	7
6	355-53-5943	A	Seoul	Member	Female	Electronic accessories	68.84	6
7	315-22-5665	C	Gyeonggi	Normal	Female	Home and lifestyle	73.56	10
8	665-32-9167	A	Seoul	Member	Female	Health and beauty	36.26	2
9	692-92-5582	B	Incheon	Member	Female	Food and beverages	54.84	3

In [87]: 1 df.iloc[2, 6]

Out[87]: 46.33

In [88]: 1 df.iloc[4:8, 1:5]

Out[88]:

	PURCHASE_PAGE	CITY	CUSTOMER_TYPE	GENDER
4	A	Seoul	Normal	Male
5	C	Gyeonggi	Normal	Male
6	A	Seoul	Member	Female
7	C	Gyeonggi	Normal	Female

데이터 행, 열 일부 추출(iloc, loc)

```
In [89]: 1 df.iloc[5:, :4]
```

Out[89]:

	ORDERID	PURCHASE_PAGE	CITY	CUSTOMER_TYPE
5	699-14-3026	C	Gyeonggi	Normal
6	355-53-5943	A	Seoul	Member
7	315-22-5665	C	Gyeonggi	Normal
8	665-32-9167	A	Seoul	Member
9	692-92-5582	B	Incheon	Member
...	...	...	...	...
995	233-67-5758	C	Gyeonggi	Normal
996	303-96-2227	B	Incheon	Normal
997	727-02-1313	A	Seoul	Member
998	347-56-2442	A	Seoul	Normal
999	849-09-3807	A	Seoul	Member

995 rows x 4 columns

5: 5부터 마지막까지

:4 처음부터 4까지(4는 미포함)

데이터 행, 열 일부 추출(iloc, loc)

```
In [90]: 1 df.iloc[:, -1]
```

```
Out[90]: 0      9.1
          1      9.6
          2      7.4
          3      8.4
          4      0.0
          ...
          995    6.2
          996    4.4
          997    7.7
          998    4.1
          999    6.6
          Name: RATING, Length: 1000, dtype: float64
```

-1: 마지막 열

마지막 열로부터 4번째 부터
마지막 열로부터 2번째 열까지

```
In [91]: 1 df.iloc[:, -4:-1]
```

Out[91]:

	COGS	GROSS_MARGIN_PERCENTAGE	GROSS_INCOME
0	522.83	4.761905	26.1415
1	76.40	4.761905	3.8200
2	324.31	4.761905	16.2155
3	465.76	4.761905	23.2880
4	604.17	4.761905	30.2085
...	...	...	...
995	40.35	4.761905	2.0175
996	973.80	4.761905	48.6900
997	31.84	4.761905	1.5920
998	65.82	4.761905	3.2910
999	618.38	4.761905	30.9190

1000 rows x 3 columns

데이터 행, 열 일부 추출(iloc, loc)

```
In [92]: 1 df.loc[:, "CITY"]
```

```
Out[92]: 0      Seoul
          1    Gyeonggi
          2      Seoul
          3      Seoul
          4      Seoul
          ...
          995    Gyeonggi
          996    Incheon
          997      Seoul
          998      Seoul
          999      Seoul
          Name: CITY, Length: 1000, dtype: object
```

모든 행, CITY열

```
In [93]: 1 df.loc[df['UNIT_PRICE'] > 50, 'CITY']
```

```
Out[93]: 0      Seoul
          3      Seoul
          4      Seoul
          5    Gyeonggi
          6      Seoul
          ...
          992      Seoul
          994    Gyeonggi
          996    Incheon
          998      Seoul
          999      Seoul
          Name: CITY, Length: 561, dtype: object
```

'UNIT_PRICE' > 50인 데이터의 CITY열

새로운 열(column) 추가하기

```
In [94]: 1 df['PRICE_N_MILEAGE'] = df['TOTAL_PRICE'] + df['MILEAGE']
```

```
In [95]: 1 df.head(5)
```

Out[95]:

PRICE	QUANTITY	MILEAGE	TOTAL_PRICE	DATE	PURCHASE_TIME	PAYMENT	COGS	GROSS_MARGIN_PERCENTAGE	GROSS_INCOME	RATING	PRICE_N_MILEAGE
4.69	7	26.1415	548.9715	1/5/2019	13:08	Samsungpay	522.83	4.761905	26.1415	9.1	575.113
5.28	5	3.8200	80.2200	3/8/2019	10:29	Naverpay	76.40	4.761905	3.8200	9.6	84.040
6.33	7	16.2155	340.5255	3/3/2019	13:23	Credit card	324.31	4.761905	16.2155	7.4	356.741
8.22	8	23.2880	489.0480	1/27/2019	20:33	Samsungpay	465.76	4.761905	23.2880	8.4	512.336
6.31	7	30.2085	634.3785	2/8/2019	10:37	Samsungpay	604.17	4.761905	30.2085	0.0	664.587

TOTAL_PRICE열과 MILEAGE 열을 더한 새로운 열 PRICE_N_MILEAGE 열 추가

열(column) 삭제하기

```
In [96]: 1 df.drop('PRICE_N_MILEAGE', axis=1, inplace=True)
```

열 방향 기존 데이터 대체

```
In [97]: 1 df.head(5)
```

Out[97]:

DUCT_TYPE	UNIT_PRICE	QUANTITY	MILEAGE	TOTAL_PRICE	DATE	PURCHASE_TIME	PAYMENT	COGS	GROSS_MARGIN_PERCENTAGE	GROSS_INCOME	RATING
health and beauty	74.69	7	26.1415	548.9715	1/5/2019	13:08	Samsungpay	522.83	4.761905	26.1415	9.1
lectronic cessories	15.28	5	3.8200	80.2200	3/8/2019	10:29	Naverpay	76.40	4.761905	3.8200	9.6
Home and lifestyle	46.33	7	16.2155	340.5255	3/3/2019	13:23	Credit card	324.31	4.761905	16.2155	7.4
health and beauty	58.22	8	23.2880	489.0480	1/27/2019	20:33	Samsungpay	465.76	4.761905	23.2880	8.4
ports and travel	86.31	7	30.2085	634.3785	2/8/2019	10:37	Samsungpay	604.17	4.761905	30.2085	0.0

새로운 열(column) 추가하기

▶

1 df.describe()

9]:

	UNIT_PRICE	QUANTITY	MILEAGE	TOTAL_PRICE	COGS	GROSS_MARGIN_PERCENTAGE	GROSS_INCOME	RATING
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000
mean	55.672130	5.510000	15.379369	322.966749	307.58738	4.761905	15.379369	6.949100
std	26.494628	2.923431	11.708825	245.885335	234.17651	0.000000	11.708825	1.769715
min	10.080000	1.000000	0.508500	10.678500	10.17000	4.761905	0.508500	0.000000
25%	32.875000	3.000000	5.924875	124.422375	118.49750	4.761905	5.924875	5.500000
50%	55.230000	5.000000	12.088000	253.848000	241.76000	4.761905	12.088000	6.950000
75%	77.935000	8.000000	22.445250	471.350250	448.90500	4.761905	22.445250	8.500000
max	99.960000	10.000000	49.650000	1042.650000	993.00000	4.761905	49.650000	10.000000

새로운 열(column) 추가하기

In [81]: ▶

1 n = len(df)

2 print(n)

1000

In [82]: ▶

1 df['GRADE'] = ''

In [83]: ▶

1 df.head(5)

Out[83]:

TYPE	UNIT_PRICE	QUANTITY	MILEAGE	TOTAL_PRICE	DATE	PURCHASE_TIME	PAYMENT	COGS	GROSS_MARGIN_PERCENTAGE	GROSS_INCOME	RATING	GRADE
ind ity	74.69	7	26.1415	548.9715	1/5/2019	13:08	Samsungpay	522.83	4.761905	26.1415	9.1	
ic es	15.28	5	3.8200	80.2200	3/8/2019	10:29	Naverpay	76.40	4.761905	3.8200	9.6	
ind le	46.33	7	16.2155	340.5255	3/3/2019	13:23	Credit card	324.31	4.761905	16.2155	7.4	
ind ity	58.22	8	23.2880	489.0480	1/27/2019	20:33	Samsungpay	465.76	4.761905	23.2880	8.4	
ind rel	86.31	7	30.2085	634.3785	2/8/2019	10:37	Samsungpay	604.17	4.761905	30.2085	0.0	

새로운 열(column) 추가하기

In [84]: ▶

```
1 for i in range(n):
2     total_price = df['TOTAL_PRICE'][i]
3
4     if total_price < 200:
5         df.loc[i, 'GRADE'] = 'NORMAL'
6     elif 200 <= total_price < 500:
7         df.loc[i, 'GRADE'] = 'VIP'
8     else:
9         df.loc[i, 'GRADE'] = 'VVIP'
```

새로운 열(column) 추가하기

In [85]: ▶

1 df

Out[85]:

ID	UNIT_PRICE	QUANTITY	MILEAGE	TOTAL_PRICE	DATE	PURCHASE_TIME	PAYMENT	COGS	GROSS_MARGIN_PERCENTAGE	GROSS_INCOME	RATING	GRADE
id :y	74.69	7	26.1415	548.9715	1/5/2019	13:08	Samsungpay	522.83	4.761905	26.1415	9.1	VVIP
.c :s	15.28	5	3.8200	80.2200	3/8/2019	10:29	Naverpay	76.40	4.761905	3.8200	9.6	NORMAL
id .e	46.33	7	16.2155	340.5255	3/3/2019	13:23	Credit card	324.31	4.761905	16.2155	7.4	VIP
id :y	58.22	8	23.2880	489.0480	1/27/2019	20:33	Samsungpay	465.76	4.761905	23.2880	8.4	VIP
id :l	86.31	7	30.2085	634.3785	2/8/2019	10:37	Samsungpay	604.17	4.761905	30.2085	0.0	VVIP
.	...	...	...	...	...	...	...	...	...	...	...	...
id :y	40.35	1	2.0175	42.3675	1/29/2019	13:46	Samsungpay	40.35	4.761905	2.0175	6.2	NORMAL
id .e	97.38	10	48.6900	1022.4900	3/2/2019	17:16	Samsungpay	973.80	4.761905	48.6900	4.4	VVIP
id :s	31.84	1	1.5920	33.4320	2/9/2019	13:22	Naverpay	31.84	4.761905	1.5920	7.7	NORMAL
id .e	65.82	1	3.2910	69.1110	2/22/2019	15:33	Naverpay	65.82	4.761905	3.2910	4.1	NORMAL
on :s	88.34	7	30.9190	649.2990	2/18/2019	13:28	Naverpay	618.38	4.761905	30.9190	6.6	VVIP

날짜 다루기

```
[39]: import datetime

now = datetime.datetime.now()
print(now)

2025-03-10 09:07:16.023749
```

```
[40]: nowDatetimeS = now.strftime('%Y-%m-%d %H:%M:%S.%f')
print(nowDatetimeS)

2025-03-10 09:07:16.023749
```

```
[41]: nowDatetime = now.strftime('%Y-%m-%d %H:%M:%S')
print(nowDatetime)

2025-03-10 09:07:16
```

```
[42]: nowDate = now.strftime('%Y-%m-%d')
print(nowDate)

2025-03-10
```

```
[43]: nowHM = now.strftime('%H:%M')
print(nowHM)

09:07
```

```
[44]: nowHM2 = now.strftime('%H%M')
print(nowHM2)

0907
```

날짜 다루기

[45]:
dt = now - datetime.timedelta(minutes=20)
print(dt)
2025-03-10 08:47:16.023749

지금으로부터 20분전 시간 구하기

단위	문법
1주	datetime.timedelta(weeks=1)
1일	datetime.timedelta(days=1)
1시간	datetime.timedelta(hours=1)
1분	datetime.timedelta(minutes=1)
1초	datetime.timedelta(seconds=1)
1밀리초	datetime.timedelta(milliseconds=1)
1마이크로초	datetime.timedelta(microseconds=1)

날짜 다루기

```
In [88]: 1 df['DATE'] = pd.to_datetime(df['DATE'], format='%m/%d/%Y')
                                     월 일 연도
```

```
In [89]: 1 df.head(5)
```

Out[89]:

	ORDERID	PURCHASE_PAGE	CITY	CUSTOMER_TYPE	GENDER	PRODUCT_TYPE	UNIT_PRICE	QUANTITY	MILEAGE	TOTAL_PRICE	DATE	PURCHASE_TIME	PAY
0	750-67-8428	A	Seoul	Member	Female	Health and beauty	74.69	7	26.1415	548.9715	2019-01-05	13:08	Samsur
1	226-31-3081	C	Gyeonggi	Normal	Female	Electronic accessories	15.28	5	3.8200	80.2200	2019-03-08	10:29	Nave
2	631-41-3108	A	Seoul	Normal	Male	Home and lifestyle	46.33	7	16.2155	340.5255	2019-03-03	13:23	Cr
3	123-19-1176	A	Seoul	Member	Male	Health and beauty	58.22	8	23.2880	489.0480	2019-01-27	20:33	Samsur
4	373-73-7910	A	Seoul	Normal	Male	Sports and travel	86.31	7	30.2085	634.3785	2019-02-08	10:37	Samsur

2025년 12월 25일과 DATE의 차이

```
[54]: from datetime import datetime

xmas = datetime.strptime('2025-12-25', '%Y-%m-%d').date()
print(xmas)

2025-12-25
```

```
[55]: df['TO_XMAS'] = (xmas - df['DATE'].dt.date).astype(str).str[:5].astype(int)
```

```
[56]: type(xmas)
```

```
[56]: datetime.date
```

```
[58]: df.head(5)
```

PRICE	QUANTITY	MILEAGE	TOTAL_PRICE	DATE	PURCHASE_TIME	PAYMENT	COGS	GROSS_MARGIN_PERCENTAGE	GROSS_INCOME	RATING	GRADE	TO_XMAS
74.69	7	26.1415	548.9715	2019-01-05	13:08	Samsungpay	522.83	4.761905	26.1415	9.1	VVIP	2546
15.28	5	3.8200	80.2200	2019-03-08	10:29	Naverpay	76.40	4.761905	3.8200	9.6	NORMAL	2484
46.33	7	16.2155	340.5255	2019-03-03	13:23	Credit card	324.31	4.761905	16.2155	7.4	VIP	2489
58.22	8	23.2880	489.0480	2019-01-27	20:33	Samsungpay	465.76	4.761905	23.2880	8.4	VIP	2524
86.31	7	30.2085	634.3785	2019-02-08	10:37	Samsungpay	604.17	4.761905	30.2085	0.0	VVIP	2512

정렬(오름차순)

In [93]:

▶

1 df010 = df.sort_values('PURCHASE_TIME', ascending=True)

In [94]:

▶

1 df010.head(10)

Out[94]:

IIT_PRICE	QUANTITY	MILEAGE	TOTAL_PRICE	DATE	PURCHASE_TIME	PAYMENT	COGS	GROSS_MARGIN_PERCENTAGE	GROSS_INCOME	RATING	GRADE	TO_XMAS
51.34	8	20.5360	431.2560	2019-01-31	10:00	Samsungpay	410.72	4.761905	20.5360	7.6	VIP	2155
64.26	7	22.4910	472.3110	2019-02-09	10:00	Naverpay	449.82	4.761905	22.4910	5.7	VIP	2146
36.36	2	3.6360	76.3560	2019-01-21	10:00	Naverpay	72.72	4.761905	3.6360	7.1	NORMAL	2165
19.15	6	5.7450	120.6450	2019-01-29	10:01	Credit card	114.90	4.761905	5.7450	6.8	NORMAL	2157
40.62	2	4.0620	85.3020	2019-01-17	10:01	Credit card	81.24	4.761905	4.0620	4.1	NORMAL	2169
61.41	7	21.4935	451.3635	2019-01-14	10:02	Naverpay	429.87	4.761905	21.4935	9.8	VIP	2172
73.95	4	14.7900	310.5900	2019-02-03	10:02	Naverpay	295.80	4.761905	14.7900	6.1	VIP	2152
21.58	1	1.0790	22.6590	2019-02-09	10:02	Samsungpay	21.58	4.761905	1.0790	7.2	NORMAL	2146
45.58	7	15.9530	335.0130	2019-01-13	10:03	Naverpay	319.06	4.761905	15.9530	5.0	VIP	2173
78.77	10	39.3850	827.0850	2019-01-24	10:04	Naverpay	787.70	4.761905	39.3850	6.4	VVIP	2162

정렬(내림차순)

```
In [95]: 1 df011 = df.sort_values('PURCHASE_TIME', ascending=False)
```

```
In [97]: 1 df011
```

Out[97]:

IIT_PRICE	QUANTITY	MILEAGE	TOTAL_PRICE	DATE	PURCHASE_TIME	PAYMENT	COGS	GROSS_MARGIN_PERCENTAGE	GROSS_INCOME	RATING	GRADE	TO_XMAS
85.72	3	12.8580	270.0180	2019-01-24	20:59	Samsungpay	257.16	4.761905	12.8580	5.1	VIP	2162
26.31	5	6.5775	138.1275	2019-01-18	20:59	Credit card	131.55	4.761905	6.5775	8.8	NORMAL	2168
28.38	5	7.0950	148.9950	2019-03-06	20:57	Naverpay	141.90	4.761905	7.0950	9.4	NORMAL	2121
74.60	10	37.3000	783.3000	2019-01-08	20:55	Naverpay	746.00	4.761905	37.3000	9.5	VVIP	2178
24.18	8	9.6720	203.1120	2019-01-28	20:54	Samsungpay	193.44	4.761905	9.6720	9.8	VIP	2158
...	...	...	...	...	...	...	...	...	...	...	...	...
19.15	6	5.7450	120.6450	2019-01-29	10:01	Credit card	114.90	4.761905	5.7450	6.8	NORMAL	2157
40.62	2	4.0620	85.3020	2019-01-17	10:01	Credit card	81.24	4.761905	4.0620	4.1	NORMAL	2169
64.26	7	22.4910	472.3110	2019-02-09	10:00	Naverpay	449.82	4.761905	22.4910	5.7	VIP	2146
51.34	8	20.5360	431.2560	2019-01-31	10:00	Samsungpay	410.72	4.761905	20.5360	7.6	VIP	2155
36.36	2	3.6360	76.3560	2019-01-21	10:00	Naverpay	72.72	4.761905	3.6360	7.1	NORMAL	2165

AI/BigData 인텐시브 과정

Section 2. 데이터 요약

Section 2-1. groupby()

groupby() 데이터 그룹화하기

```
In [98]: 1 df.groupby('GENDER')['TOTAL_PRICE'].mean()
```

```
Out[98]: GENDER
Female    335.095659
Male      310.789226
Name: TOTAL_PRICE, dtype: float64
```

```
In [99]: 1 df.groupby('GENDER')['TOTAL_PRICE'].median()
```

```
Out[99]: GENDER
Female    272.58
Male      244.23
Name: TOTAL_PRICE, dtype: float64
```

```
In [100]: 1 df.groupby('GENDER', as_index=False)['TOTAL_PRICE'].mean()
```

Out[100]:

	GENDER	TOTAL_PRICE
0	Female	335.095659
1	Male	310.789226

```
In [101]: 1 df.groupby('GENDER', as_index=False)['TOTAL_PRICE'].median()
```

Out[101]:

	GENDER	TOTAL_PRICE
0	Female	272.58
1	Male	244.23

AI/BigData 인텐시브 과정

Section 2. 데이터 요약

Section 2-2. crosstab()

crosstab

```
In [102]: ▶ 1 table1 = pd.crosstab(df['GENDER'], df['CITY'])
```

```
In [103]: ▶ 1 table1
```

Out[103]:

	CITY		
	Gyeonggi	Incheon	Seoul
GENDER			
Female	178	162	161
Male	150	170	179

crosstab

```
In [104]: 1 table2 = pd.crosstab(df['GENDER'], df['CITY'], normalize='index')
```

행 백분율

```
In [105]: 1 table2
```

Out[105]:

	CITY	Gyeonggi	Incheon	Seoul
GENDER				
Female	0.355289	0.323353	0.321357	
Male	0.300601	0.340681	0.358717	

```
In [106]: 1 table3 = pd.crosstab(df['GENDER'], df['CITY'], normalize='columns')
```

열 백분율

```
In [107]: 1 table3
```

Out[107]:

	CITY	Gyeonggi	Incheon	Seoul
GENDER				
Female	0.542683	0.487952	0.473529	
Male	0.457317	0.512048	0.526471	

crosstab

```
In [108]: ▶ 1 table4 = pd.crosstab(df['GENDER'], df['CITY'], margins=True)
```

```
In [109]: ▶ 1 table4
```

Out[109]:

	CITY	Gyeonggi	Incheon	Seoul	All
GENDER					
Female		178	162	161	501
Male		150	170	179	499
All		328	332	340	1000

AI/BigData 인텐시브 과정

Section 2. 데이터 요약

Section 2-3. `pivot_table()`

pivot_table()

```
In [110]: ▶ 1 pivot1 = df.pivot_table(index='GENDER', columns='CITY', values='TOTAL_PRICE', aggfunc='sum')  
          2 pivot1
```

합산 함수

Out[110]:

	CITY	Gyeonggi	Incheon	Seoul
GENDER				
Female	61685.4630	52928.295	53269.1670	
Male	48883.2435	53269.377	52931.2035	

pivot_table()

```
In [111]: ▶ 1 pivot2 = df.pivot_table(index='GENDER', columns='CITY', values='TOTAL_PRICE', aggfunc='mean')  
          2 pivot2
```

Out[111]:

	CITY	Gyeonggi	Incheon	Seoul
GENDER				
Female	346.547545	326.717870	330.864391	
Male	325.888290	313.349276	295.705047	

```
In [112]: ▶ 1 pivot3 = df.pivot_table(index='GENDER', columns='CITY', values='TOTAL_PRICE', aggfunc='median')  
          2 pivot3
```

Out[112]:

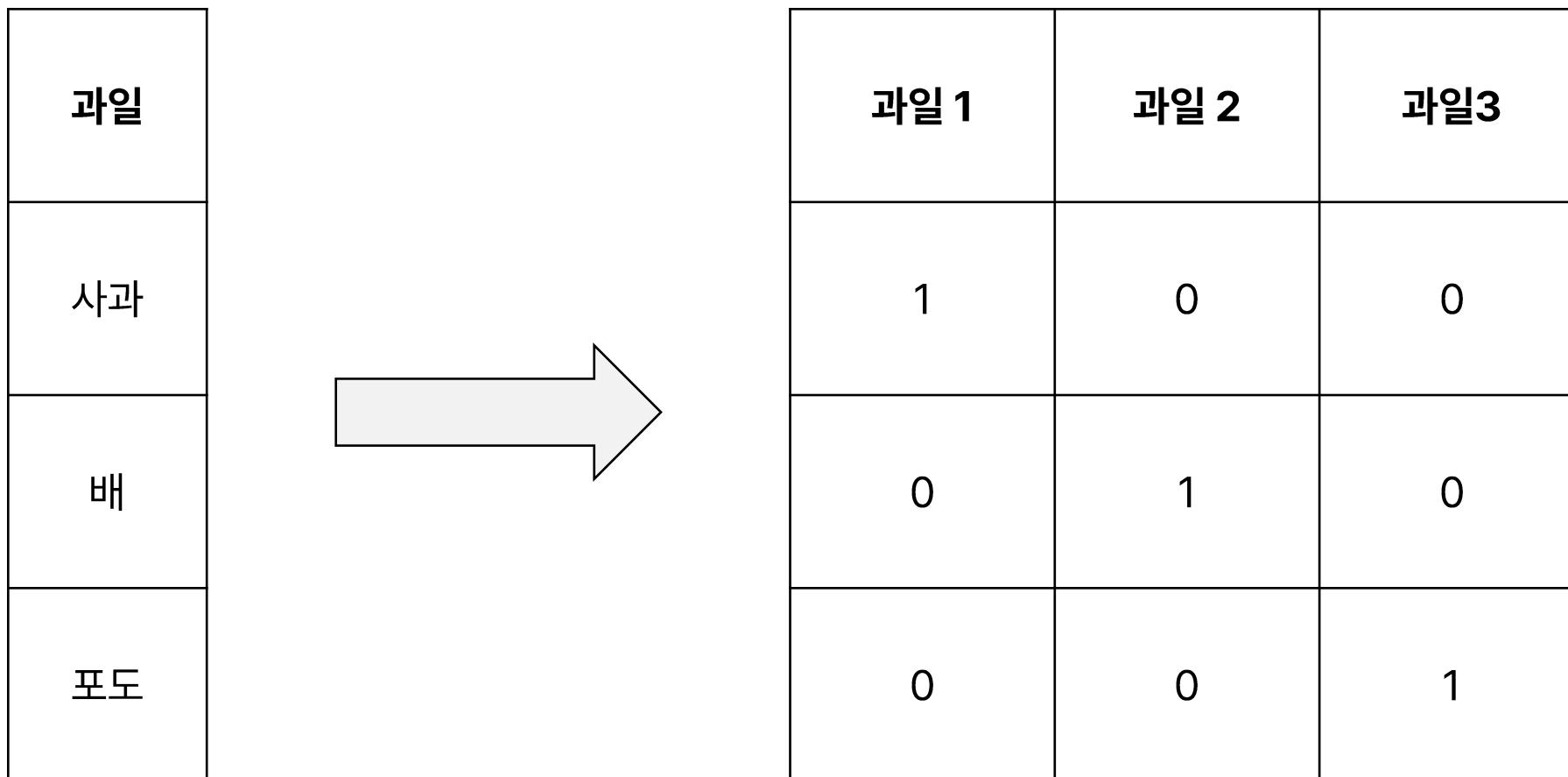
	CITY	Gyeonggi	Incheon	Seoul
GENDER				
Female	292.32000	261.32400	257.145	
Male	255.39675	249.65325	233.100	

AI/BigData 인텐시브 과정

Section 2. 데이터 요약

Section 2-4. one hot encoding

one hot encoding



one hot encoding

```
In [1]: 1 import numpy as np
        2 import pandas as pd
```

```
In [4]: 1 df = pd.DataFrame([
        2     [42, 'male', 12, 'reading', 1],
        3     [35, 'female', 3, 'cooking', 0],
        4     [27, 'female', 7, 'cycling', 2],
        5     [56, 'male', 21, 'swimming', 0]
        6 ])
        7
        8 df.columns=['age',
        9             'gender',
       10             'month_birth',
       11             'hobby',
       12             'target']
```

```
In [5]: 1 df
```

Out[5]:

	age	gender	month_birth	hobby	target
0	42	male	12	reading	1
1	35	female	3	cooking	0
2	27	female	7	cycling	2
3	56	male	21	swimming	0

one hot encoding

```
In [12]: 1 df_oh = pd.get_dummies(df, dtype=float)
```

```
In [13]: 1 df_oh
```

Out[13]:

	age	month_birth	target	gender_female	gender_male	hobby_cooking	hobby_cycling	hobby_reading	hobby_swimming
0	42	12	1	0.0	1.0	0.0	0.0	1.0	0.0
1	35	3	0	1.0	0.0	1.0	0.0	0.0	0.0
2	27	7	2	1.0	0.0	0.0	1.0	0.0	0.0
3	56	21	0	0.0	1.0	0.0	0.0	0.0	1.0

```
In [18]: 1 pd.get_dummies(df['hobby'], dtype=float)
```

Out[18]:

	cooking	cycling	reading	swimming
0	0.0	0.0	1.0	0.0
1	1.0	0.0	0.0	0.0
2	0.0	1.0	0.0	0.0
3	0.0	0.0	0.0	1.0

감사합니다.

Q & A